ServiceIQ Agent — Solution Architecture (Phase 2 & 3)

Oct 28, 2026 · @Punit

ServiceIQ Phase 2 and 3 add **one LangGraph agent** that turns a homeowner's chat request into a confirmed appointment and work order in under 5 minutes, replacing about 15 minutes of manual work per job. It reuses the MVP's Q&A pipeline as a read-only tool and pauses for a person before any decision that carries money, warranty or safety risk.

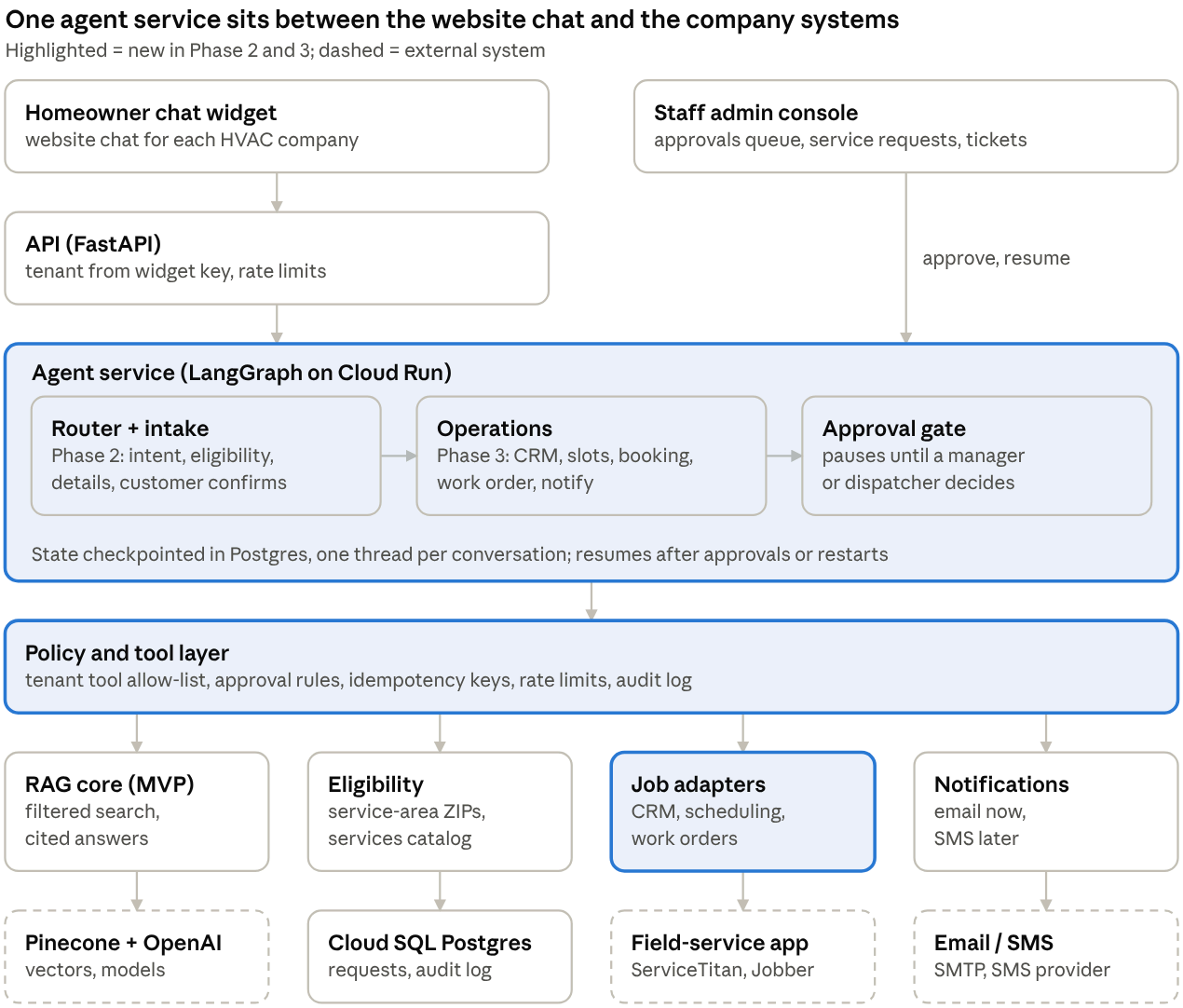
Agent at a glance

Field	Design
Goal	Turn a homeowner's service request into a confirmed appointment and work order.
Channel	Website chat widget for homeowners; ServiceIQ admin console for reps, dispatchers and managers.
Steps	Detect intent, check eligibility, collect details, customer confirms, book or route to approval, confirm with customer.
Tools	3 that only read (knowledge search, service-area check, availability) and 4 that change something (service request, CRM record, booking and work order, notification).
Memory	Conversation details in graph state; request ID and workflow state checkpointed in Postgres across sessions.
Never	Binding quotes, discounts, warranty decisions, after-hours dispatch on its own, repair steps for gas, electrical or refrigerant, payments, cross-tenant data.
Human review	Customer confirms the summary; manager or dispatcher approves policy exceptions; dispatchers can override any booking.

Field	Design
On failure	One idempotent retry, then save the request as a ticket for a rep and tell the customer; emergencies always get the fixed safety script.
Success	Standard requests booked and confirmed in under 5 minutes with no re-keying; dispatchers keep 9 of 10 bookings unchanged.

Solution architecture

The agent runs as one new service behind the existing API, and reaches every outside system only through the policy and tool layer.



solution architecture · channels, agent service, policy layer, tools, systems

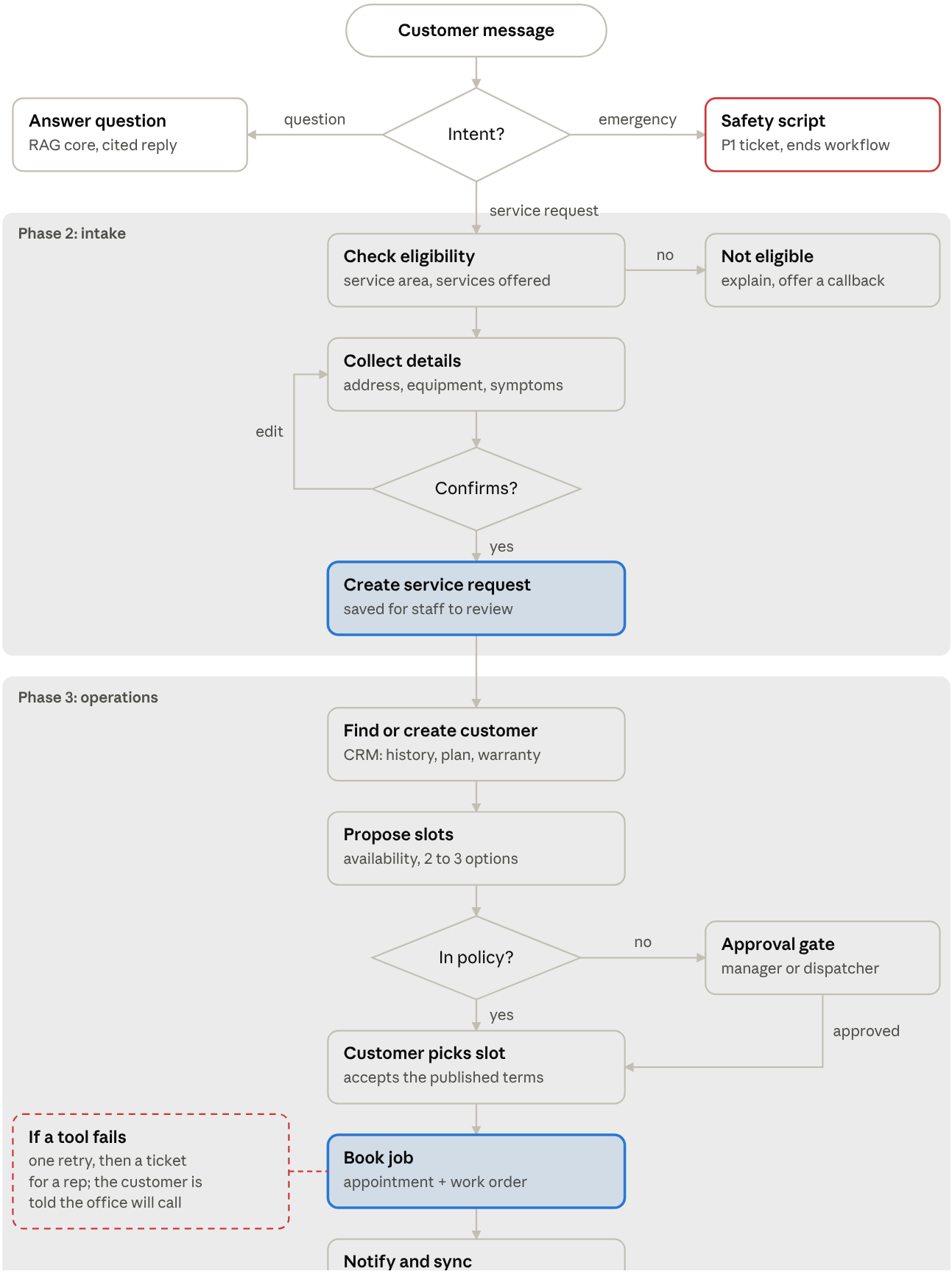
Customer messages enter through the API, which fixes the tenant from the widget key; staff approvals from the admin console resume a paused conversation directly in the agent service.

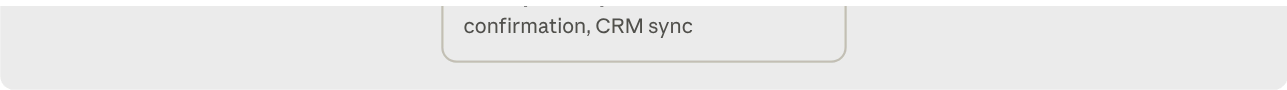
Agent workflow

Every message passes through one graph: questions reuse the MVP's answers, emergencies leave for the fixed safety script, and service requests run intake, then operations.

Only standard, in-policy jobs are booked without a person

One LangGraph graph; emergencies can exit at any step





agent workflow · router, Phase 2 intake, Phase 3 operations, approval gate

The two highlighted steps are the only points where the agent creates records for the company; anything outside the approval policy waits at the gate, and the graph resumes from its Postgres checkpoint once someone decides.

Tools and approval policy

The agent gets 7 tools, and only 4 of them change anything. Every tool call goes through one policy layer that checks the tenant's allow-list, approval rules, rate limits and an idempotency key before the adapter runs.

Tool	Kind	Backed by	Guard before it runs
`search_knowledge`	Read	MVP RAG core: filtered Pinecone search, grounded answer with citations	None; same guardrails as the Q&A system
`check_eligibility`	Read	Exact ZIP lookup in the service-area table plus the services catalog	None
`get_availability`	Read	Scheduling adapter: open slots by skill and territory	None
`create_service_request`	Write	ServiceIQ Postgres	Customer confirms the summary
`upsert_customer`	Read and write	CRM adapter: find or create the customer, equipment, plan	Auto; matched on phone and address
`book_job`	Write	Scheduling and work-order adapters: books the slot and creates the work order	Auto-approval policy below
`notify_customer`	Write	Notification adapter: email now, SMS later	Approved templates only

Auto-approval policy (from the architecture document, section 3.7)

Situation	Agent acts alone?	Otherwise approved by
Standard visit, business hours, published price	Yes	—
Binding quote, discount or price outside the published range	No	Service manager
After-hours or emergency dispatch	No	On-call dispatcher
Warranty or contract coverage decision	No	Service manager
Customer reschedule or cancel within policy	Yes	Rep, if outside policy

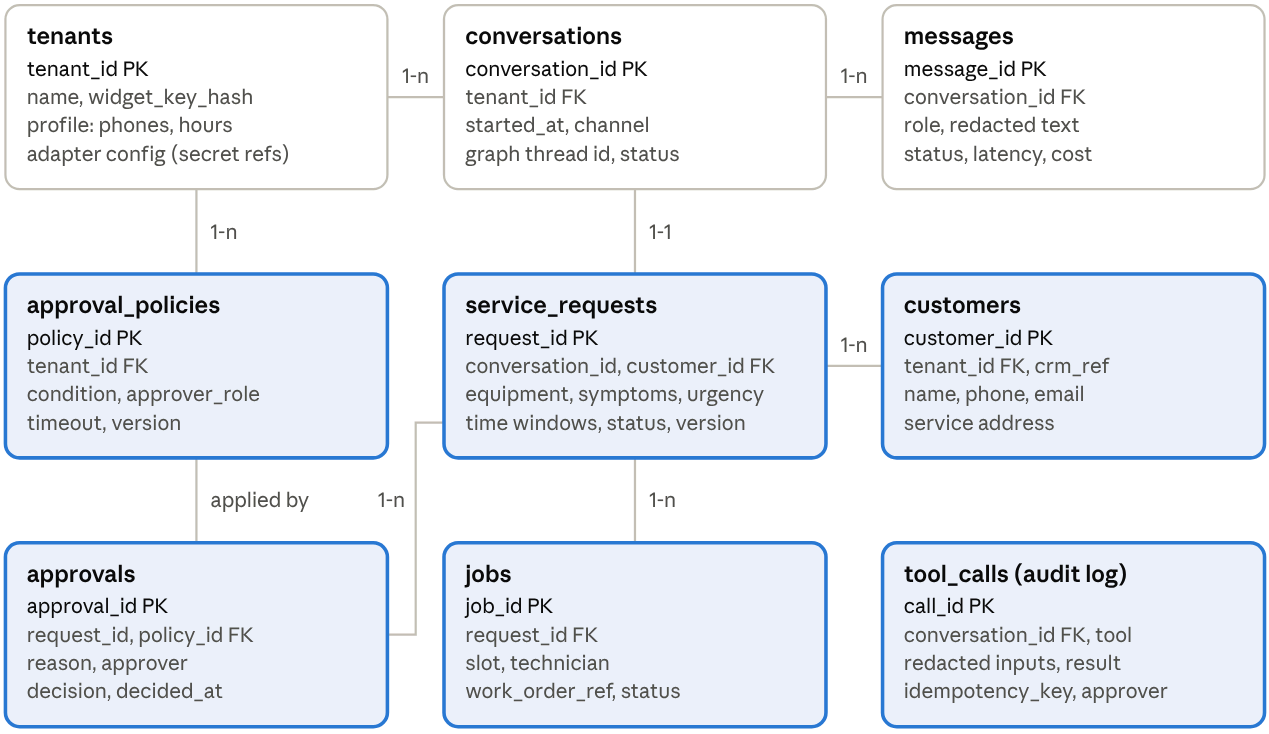
The policy is data, stored per tenant, so a company can tighten it without a code change.

Data model

Phase 2 and 3 add six tables to the MVP's Postgres database; the service request is the hub that ties a conversation to a customer, approvals and the booked job.

Each service request links a conversation to its customer, approvals and job

Highlighted = new tables in Phase 2 and 3; others reused from the MVP



Also reused: escalations (tickets), document registry, admin users.
LangGraph keeps its checkpoint tables in the same Postgres database.

data model · 3 reused and 6 new tables

`service_requests` follows a versioned JSON schema, so the admin console and future adapters read the same shape. Customer contact details live only in `customers` and the field-service system; messages and tool-call inputs store redacted text.

System stack

Most of the stack is reused from the MVP; the new parts are the agent runtime, the tool adapters, the policy layer and the approval queue.

Layer	Technology	Status
Customer channel	Streamlit chat for the pilot; embeddable JavaScript widget per tenant	Reused; widget new
Staff console	Streamlit admin console with a new approvals queue and service-request view	Extended

Layer	Technology	Status
API	FastAPI; tenant resolved from the hashed widget key; rate limits; CORS	Reused, new `/v1/agent` routes
Agent runtime	Python, LangGraph state graph with subgraphs and interrupt-based approval pauses	New
Agent state	LangGraph Postgres checkpointer, one thread per conversation	New
Language models	`gpt-4.1-mini` for routing and detail extraction; `gpt-5.4-mini` for customer-facing replies and tool calls (chosen in M10)	Reused models
Knowledge	MVP RAG core: guardrails, filtered Pinecone search, grounded generation, number checks	Reused as a read-only tool
Vector store	Pinecone serverless, one namespace per tenant, `text-embedding-3-small`	Reused
Database	PostgreSQL (Cloud SQL), SQLAlchemy	Reused, new tables
Integrations	Adapter interface per system (CRM, scheduling, work orders, notifications) with a mock adapter for dev and tests	New
Background work	Cloud Tasks for retries, reminders and approval timeouts	New
Notifications	SMTP email now; SMS provider later	Email reused; SMS new
Observability	LangSmith traces per turn and per tool call; audit-log table	Reused, audit log new
Hosting and secrets	Docker image, Google Cloud Run, Secret Manager, Cloud Storage	Reused

Reliability, security and observability

The agent fails toward a person: when anything it can't verify goes wrong, it saves what it has as a ticket and tells the customer the office will call.

Failure handling

Failure	What the agent does
Tool error or timeout	One retry with the same idempotency key, so a booking is never made twice
Retry fails or adapter is down	Stop, save the request as a ticket for a rep, tell the customer the office will call
Required detail missing or invalid	Ask the customer again, at most twice, then hand off
Approval not given within the tenant's time limit	Escalate to the next approver; remind the customer the request is in review
Emergency detected at any step	Leave the workflow and return the fixed safety script; open a P1 ticket
Process restart mid-conversation	Resume from the last Postgres checkpoint

Security and guardrails

- **Tenant isolation:** the tenant comes only from the widget key, never from chat text; every tool and query is scoped to it.
- **Least privilege:** each tenant has a tool allow-list and its own adapter credentials in Secret Manager.
- **Untrusted input:** customer messages and tool results are treated as data; the MVP's injection rules and classifier run on every turn.
- **Personal data:** phones, emails and card numbers are replaced with placeholders before text reaches models or traces, and the real values go straight to the database; conversations are purged after 90 days.
- **Hard limits in code, not prompts:** quotes, discounts, warranty decisions and after-hours dispatch are blocked by the policy layer even if the model asks for them.

Observability

- Every tool call writes an audit-log row: who, which tool, inputs, result, approver.

- LangSmith traces each turn and tool call; the admin console shows time-to-book, approvals waiting and failed tool calls.

Deployment and delivery plan

The agent ships as two new Cloud Run services, `agent` and `worker`, beside the MVP's `api`, `chat` and `admin`, using the same Docker image with new `agent` and `worker` roles.

Service	Runs	Scales	Reaches
`api`	FastAPI: widget traffic, tenant resolution, rate limits	On requests	`agent`, Postgres
`agent` (new)	LangGraph graph and tool layer	On requests; private	RAG core, adapters, Postgres, OpenAI
`chat`	Customer chat (pilot)	On requests	`api` with a Google ID token
`admin`	Staff console with approvals queue	On requests	Postgres, `agent` to resume paused threads
`worker` (new)	Cloud Tasks handlers: retries, reminders, approval timeouts	On tasks	`agent`, adapters
Cloud SQL Postgres	Registry, tickets, requests, checkpoints, audit log	Managed	—

Dev uses Docker Compose with mock adapters; staging uses a sandbox account of the first field-service platform; production adds that platform's live credentials per tenant.

Build order, each step tested and reviewed before the next, as in the MVP:

1. Agent service skeleton: router, intake subgraph, `search_knowledge` and `check_eligibility` tools, Postgres checkpointer.
2. Intake evaluation set: scripted multi-turn dialogues with expected service requests, since the MVP data has none.
3. Service-request creation, customer confirmation, and the request view in the admin console (Phase 2 complete).
4. Adapter interface with mock CRM, scheduling, work-order and notification adapters.
5. Policy layer, approval gate, approvals queue and audit log.

6. First real adapter set in a sandbox, then a pilot with one tenant (Phase 3 complete).

Success measures and open questions

The agent succeeds when standard requests go from first message to a booked, confirmed appointment in under 5 minutes with no re-keying, and dispatchers keep 9 of 10 bookings unchanged.

Measure	Target	Source
Time from first message to confirmed booking (standard requests)	Under 5 minutes, down from about 15	Agent timestamps
Bookings kept unchanged by dispatchers	9 of 10	Audit log overrides
Inquiries resolved or turned into complete requests without a rep	At least 40%	Requirements pilot hypothesis
Positive customer feedback	At least 85%	Requirements pilot hypothesis
Actions outside the approval policy	0	Audit log
Emergencies routed to the safety script	100%	Evaluation set

Open questions

- [ ] Which field-service platform comes first: ServiceTitan, Housecall Pro or Jobber? (architecture document Q6)
- [ ] Which channels are required at launch beyond email: SMS, Slack or Teams? (Q4)
- [ ] How long may an approval wait before it escalates, and to whom, per tenant?
- [ ] Does each tenant accept the proposed auto-approval policy as written?